

NextFem Marketplace

Manual t cnico de NextFem

Gu a formal para instalaci n, mantenimiento, arquitectura y despliegue.

Tipo de documento	Manual t�cnico
Proyecto	Marketplace gratuito para mujeres emprendedoras
Estado	Versi�n funcional con frontend, backend, base de datos y despliegue preparado
Fecha	Julio de 2026

1. Prop sito del manual

Este manual t cnico describe la estructura interna de NextFem Marketplace y sirve como gu a para personal encargado de mantenimiento, correcci n de errores, despliegue y evoluci n del sistema.

2. Requisitos t cnicos

Recurso	Versi�n o requisito	Observaciones
Node.js	20 o superior	Definido en package.json raiz.
npm	Incluido con Node.js	En PowerShell puede utilizarse npm.cmd si npm.ps1 est� bloqueado.
PostgreSQL	Base relacional	Debe existir DATABASE_URL valida.
Navegador moderno	Chrome, Edge, Brave u otro compatible	Necesario para camara/galer�a, microfono y ubicaci�n.
Railway	Opcional para producci�n	Backend y PostgreSQL.
Vercel	Opcional para producci�n	Frontend Vite.

3. Estructura del repositorio

Ruta	Descripci�n
backend	Servidor Node.js con Express, rutas API, middlewares, Socket.IO, configuraci�n de base de datos y utilidades.
frontend	Aplicaci�n Vue con Vite, componentes, vistas, router, servicios API, composables y estilos globales.
bd	Scripts SQL ejecutados en orden para crear y actualizar el esquema de PostgreSQL.
documentacion	Documentos formales, manual t�cnico y manual de usuario.
railway.json / Dockerfile	Configuraci�n de despliegue backend en Railway.

vercel.json	Configuraci�n de despliegue frontend en Vercel.
-------------	---

4. Variables de entorno

Variable	Servicio	Descripci�n
PORT	Backend	Puerto del servidor. En local normalmente 3000; en Railway se asigna autom�ticamente.
DATABASE_URL	Backend	Cadena de conexi�n PostgreSQL.
DATABASE_SSL	Backend	Activar en proveedores que requieren SSL, por ejemplo Neon o Supabase.
JWT_SECRET	Backend	Secreto para firmar tokens de sesi�n.
MESSAGE_SECRET	Backend	Secreto recomendado para cifrado de mensajes; puede caer en JWT_SECRET si no existe.
FRONTEND_URL	Backend	Origen permitido por CORS; puede aceptar varios separados por coma.
OPENAI_API_KEY	Backend	Opcional. Activa funciones reales de OpenAI.
OPENAI_MODEL	Backend	Modelo configurado para IA opcional.
OPENAI_REASONING_EFFORT	Backend	Nivel de razonamiento para llamadas de IA.
VITE_API_URL	Frontend	URL publica del backend terminando en /api.
VITE_SOCKET_URL	Frontend	URL publica del backend sin /api para Socket.IO.

5. Instalaci n local

1. Abrir una terminal en la carpeta backend y ejecutar npm.cmd install si aun no existen dependencias.
2. Configurar backend/.env con DATABASE_URL, JWT_SECRET, FRONTEND_URL y variables opcionales.
3. Ejecutar npm.cmd run db:setup para crear o actualizar las tablas.
4. Ejecutar npm.cmd run dev para iniciar el backend en http://localhost:3000.
5. Abrir otra terminal en frontend y ejecutar npm.cmd install si es necesario.

6. Ejecutar `npm.cmd run dev` para iniciar el frontend en `http://localhost:5173`.

6. Scripts principales

Ubicaci�n	Comando	Funci�n
backend	<code>npm.cmd run dev</code>	Inicia backend con nodemon para desarrollo.
backend	<code>npm.cmd run db:setup</code>	Ejecuta migraciones SQL desde la carpeta bd.
backend	<code>npm.cmd start</code>	Inicia servidor backend en modo producci�n.
frontend	<code>npm.cmd run dev</code>	Inicia servidor Vite de desarrollo.
frontend	<code>npm.cmd run build</code>	Compila frontend para producci�n.
frontend	<code>npm.cmd run preview</code>	Sirve una version compilada para revision.
raz	<code>npm scripts railway:*</code>	Comandos auxiliares usados por Railway.

7. Backend

El backend esto construido con Express y expone rutas bajo `/api`. Incluye middleware de autenticacion JWT, rate limit, manejo centralizado de errores, CORS configurable, subida de archivos y Socket.IO para eventos en tiempo real.

Ruta base	Archivo	Responsabilidad
<code>/api/auth</code>	<code>auth.routes.js</code>	Registro, login, perfil y actualizacion de usuaria.
<code>/api/products</code>	<code>products.routes.js</code>	Listado, busqueda, tendencias, similares, creacion, edici�n y eliminaci�n de productos.
<code>/api/categories</code>	<code>categories.routes.js</code>	Consulta de categor�as disponibles.
<code>/api/conversations</code>	<code>conversations.routes.js</code>	Conversaciones, mensajes, archivos, ubicacion, borrado y notificaciones.
<code>/api/engagement</code>	<code>engagement.routes.js</code>	Favoritos, reportes, bloqueos, rese�as y metricas.
<code>/api/uploads</code>	<code>uploads.routes.js</code>	Carga de imagenes, videos, audio y PDF mediante

		base64.
/api/assistant	assistant.routes.js	Funciones de IA local u OpenAI opcional.

8. Base de datos

Los scripts SQL se ejecutan en orden al correr npm.cmd run db:setup. Cada archivo agrega tablas, datos iniciales o modificaciones incrementales.

Archivo	Contenido principal
01_schema.sql	Usuarios, categorias y productos base.
02_seed.sql	Categor?as iniciales.
03_profile_chat.sql	Perfil extendido, conversaciones y mensajes.
04_demo_products.sql a 06_large_catalog.sql	Productos de demostracion y catalogo amplio.
07_engagement_ai.sql	Favoritos, reportes, rese?as y metricas base.
08_search_media_chat.sql	Eventos de busqueda, multimedia y ubicacion en mensajes.
09_notifications_message_deletion.sql	Notificaciones y borrado de mensajes.
10_user_blocks.sql	Bloqueo de usuarias.
11_product_media_reviews.sql	Galeria multimedia de productos y multiples rese?as.
12_half_star_reviews.sql	Calificacion de vendedoras de 0.5 a 5 estrellas.

9. Frontend

El frontend utiliza Vue 3 con Composition API y Vue Router. La mayor parte del estado de la aplicacion se centraliza en useMarketplace.js, que conecta vistas, servicios API, Socket.IO, sesion, productos, chat y notificaciones.

Ruta	Funci?n
src/views	Pantallas principales: Home, Auth, Publish, Profile, Chat y Courses.

src/components	Componentes reutilizables: header, navegacion movil, tarjetas, modales y confirmaciones.
src/composables/useMarketplace.js	Estado global y acciones de negocio del frontend.
src/services/api.js	Cliente HTTP, almacenamiento local, URL de API y socket.
src/router/index.js	Rutas protegidas y redirecciones.
src/styles/app.css	Diseño global, responsividad, paleta visual y layout movil.

10. Seguridad y controles

- Contraseñas protegidas con bcryptjs antes de almacenarse.
- Sesiones mediante JWT enviado en el header Authorization.
- Mensajes cifrados en la base de datos mediante secreto configurado.
- Rate limit basico para autenticacion, API, chat y subidas.
- CORS limitado a FRONTEND_URL y localhost en desarrollo.
- Reportes de productos y usuarias para moderacion.
- Bloqueo de usuarias para evitar interacci?n no deseada.
- Eliminacion de mensajes para una misma o para todas con opcion de deshacer.

11. Despliegue recomendado

7. Crear base PostgreSQL en Railway.
8. Desplegar backend en Railway y configurar DATABASE_URL, JWT_SECRET, MESSAGE_SECRET y FRONTEND_URL.
9. Verificar https://backend/healthz y https://backend/api/health.
10. Desplegar frontend en Vercel con root directory frontend.
11. Configurar VITE_API_URL con la URL del backend terminando en /api.
12. Configurar VITE_SOCKET_URL con la URL del backend sin /api.
13. Actualizar FRONTEND_URL en Railway con el dominio final de Vercel y redeplegar backend.

12. Mantenimiento recomendado

- Antes de modificar, revisar git status para identificar cambios pendientes.
- Ejecutar npm.cmd run build en frontend antes de desplegar cambios visuales o de rutas.
- Ejecutar node --check en archivos backend modificados para validar sintaxis.
- Agregar migraciones SQL incrementales en bd con numeracion consecutiva.
- No editar datos productivos directamente sin respaldo previo.
- Mantener JWT_SECRET y MESSAGE_SECRET fuera del repositorio.

- Documentar nuevas variables de entorno en README o gu a de despliegue.
- Probar registro, login, publicaci n, chat y rese as despu s de cambios mayores.

13. Problemas comunes

S�ntoma	Causa probable	Acci�n recomendada
Frontend no registra usuarios en producci�n	VITE_API_URL no apunta al backend desplegado.	Configurar variable en Vercel y redeplegar.
Backend funciona en / pero falla /api/health	Problema con DATABASE_URL o SSL.	Revisar conexi�n PostgreSQL y DATABASE_SSL.
Puerto 3000 ocupado	Otro proceso Node sigue activo.	Cerrar proceso anterior o cambiar PORT.
npm bloqueado en PowerShell	Execution Policy bloquea npm.ps1.	Usar npm.cmd.
Chat no actualiza en tiempo real	VITE_SOCKET_URL o CORS incorrectos.	Verificar URL del backend sin /api y FRONTEND_URL.
Subidas fallan	Archivo demasiado grande o formato no permitido.	Verificar l�mite, tipo MIME y endpoint /api/uploads.